Deep Learning
Assignment-I

Name :- P. Rani Susmitha

VTU no :- VTU25834

Course code :- 10212CA215

Course name :- Deep Learning

Difference between Keras, Tensorflow, Pytorch

Deep learning frameworks help build, train, and deploy machine learning models

The Three most widely used frameworks are:

1. Keras
2. Tensorflow
3. PyTorch

1. Keras

Keras is a high-level deep learning API designed for fast Prototyping and easy model building originally it was a standalone library, but now it is integrated into Tensorflow as:

characteristics

simple and user-friendly

minimal code required

Good for beginners

Strengths

very Easy to write models

clean and Readable syntax

Rapid Prototyping

Good documentation

weaknesses

Less control over lower-level operations

not ideal for highly customized architectures

slower debugging because of abstraction

Best for

Besinners in deep learning

Quick model development

standard CNN, RNN, DNN architectures

Academic Projects/ Small Prototypes

Tensorflow

TensorFlow is a full-scale deep learning framework and ecosystem developed by Google Keras is built on top of it

Tensorflow Provides

- Low-level operations
- High-level APIs
- Production deployment tools
- mobile / Edge support
- Browser support

Characteristics

- Scalable and flexible
- suitable for both research and Production
- supports distributed training

Strengths

Runs on GPU/TPU
supports deployment in mobile, web, & servers
strong community support
Great for large industry Projects
TensorBoard for visualization

weaknesses

more Complex than Keras
Lower flexibility Compared to Pytorch for research
steeper learning curve

Best for

- Industry / enterprise applications
  Deploying ML models to Production
  Large-scale training
  mobile AI applications (TF Lite)

3. PyTorch

Pytorch is a deep learning framework developed by Facebook (meta).

it is known for being:

Dynamic, flexible, and research-friendly

unlike TensorFlow (which originally used static graphs), PyTorch uses dynamic computation graphs, making it more intuitive

characteristics

Pythonic and clean

very popular in research

Easy debugging

Dynamic graph Execution

strengths

- most flexible for custom architectures
- Easy to debug using native Python tools
- Preferred for NLP and computer vision research
- Acts like regular Python code
- Dynamic graph Execution

strengths / weknesses

most flexible for custom architectures

Easy to debug using native Python tools

Preferred for NLP and computer vision

- steeper learning curve for bignees
- Fewer tools for mobile/web compared to Tensorflow
- Deployment support used to be wecker

Best for

- Research and Experimentation
- Custom and complex model architectures
- NLP, Transformers, GANs
- Academic Papers and Prototypes

2. Types of normalization in deep learning

normalization is a crucial technique in deep learning used to stabilize training, reduce internal covariate shift, improve gradient flow, and speed up convergence. Different normalization methods serve different architectures and data shapes

Types

1. Batch normalization

Batch normalization normalizes the input of each layer across the batch dimension. it computes The mean and variance of each feature across all samples in a mini-batch

Formula

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y_i = \gamma \hat{x}_i + \beta$$

Advantages

- Faster training
- Reduces internal covariate shift
- Allows higher learning rates
- Adds slight regularization effect

Disadvantages

- Dependent on batch size
- Fails when batch size is small
- Harder to use in RNNS

used in

CNN architectures

old Tranformer versions

2. Layer normalization

Layernorm normalizes across the features channels of a single sample instead of across the batch

Formula

$$\mu = \frac{1}{H}\sum_{i=1}^{H} x_i, \quad \sigma^2 = \frac{1}{H}\sum_{i=1}^{H}(x_i-\mu)^2$$

Advantages

- works well with small batch sizes
- ideal for sequence models
- Better stability for RNNs and Transformers

used in

- BERT
- GPT models
- Transformer encoder-decoders
- RNNs/LSTMs

3 Instance normalization

Instance Norm normalizes each channel of each sample individually

it is like BatchNorm but computed separately for each instance and channel

Advantages

- Removes instance-specific contrast
- Great for style transfer

used in

style transfer networks

some GAN architectures

4. Group normalization

GroupNorm divides channels into groups and normalizes within each group

Ex:- 32 channels grouped into 4 groups of 8 channels each.

Advantages

- Independent of batch size
- more stable than Batchnorm for small batch sizes
- Good balance between Layernorm and instance norm

used in

object detection models

Segmentation models (mask R-CNN)

5. weight normalization

Instead of normalizing activations, weightnorm normalizes the weights of neurons

It decomposes weights as:

$$w = \frac{g}{\|v\|} v$$

Advantages

- Helps optimizer converge faster
- makes training more stable in RNNs

Disadvantages

- Less common than Batchnorm | Layernorm
- Doesn't help as much in CNNs

used in

- Custom RNNs
- Reinforcement learning networks

6. Spectral normalization

Spectral norm normalizes the largest singular value of the weight matrix

$$\hat{W} = \frac{W}{\sigma(W)}$$

Advantages

- Extremely useful for GANs
- stabilizes discriminator training

used in

WGAN-GP

SAGAN

BigGAN

Pixel normalization

Pixel norm normalizes each pixel across all channels

Formula

$$x_{norm} = \frac{x}{\sqrt{\frac{1}{N}\sum x^2 + \epsilon}}$$

Advantages

- used in GANs to replace BatchNorm
- Eliminates dependency on both batch size and channel statistics

Used in

Progressive GANs (PGGAN)